



**T.C.
FATİH SULTAN MEHMET VAKIF UNIVERSITY**

**Faculty of Engineering Department of
Software Engineering**

2025-2026 Academic Year

Introduction to Deep Learning Final Project Report

Students Name : Zeynep Dağtekin

Student ID : 2121251007

Course Instructor : Asst. Prof. Dr. Ümit Demirbağ

Course: Introduction to Deep Learning

Programming Language : Python

Project Language: English

Submission Date: 06.06.2026

Abstract

In my project i applied three different deep learning models for Introduction to Deep Learning lecture. Selected models are Convolutional Neural Network, Long Short Term Memory and Autoencoder. In this way I worked with both image data and text data for the image part i used Intel Image Classification dataset for the text part i used Turkish Movie Sentiment Analysis Dataset.

In CNN model aim was to predict which natural scene class an image belongs to. I didn't create classes myself. Dataset already had six classes in its folder structure: buildings, forest, glacier, mountain, sea and street. Under RNN topic i used an LSTM model to predict sentiment classes from Turkish movie reviews. These classes were negative, neutral and positive. In Autoencoder model purpose was to learn a compressed representation of images and reconstruct an output close to the original image. All codes run on Kaggle Notebook by using Python and TensorFlow/Keras.

In experiments CNN model reached about 80.67% accuracy on test data. LSTM model gave about 37.34% test accuracy and test reconstruction loss of Autoencoder model was about 0.0043. According to reslts CNN model was generally successful in image classification but it showed some overfitting. Success of LSTM model was limited because Turkish reviews can be complex in meaning and the labels were created from rating scores. Autoencoder model kept the general color and shape information of the images but it produced blurry results in small details.

Introduction

Deep learning models can be used for different purposes with different types of data. For example CNN models are successful in learning local features in images, such as edges, colors, textures and shapes. RNN and LSTM models are preferred for sequential data, especially text data because they can consider the order of words. Autoencoder models try to learn a smaller representation of data without using labels and reconstruct the data from this representation.

In my project my purpose wasn't only to train the models but also to explain why each model was used for that problem and what the results mean. For this reason i selected three different tasks for that applications. CNN and LSTM models were designed as supervised classification problems. Autoencoder was designed as a reconstruction problem. In this way the project includes three different models, at least one text data application and at least one image data application as required in the assignment.

Used Datasets and General Method

In my project two open access datasets were used. Intel Image Classification dataset was used for two different tasks: image classification with CNN and image reconstruction with Autoencoder. Turkish Movie Sentiment Analysis Dataset was used for Turkish sentiment analysis with an RNN-based LSTM model.

Dataset	Used Model	Data Type	Description
Intel Image Classification	CNN ve Autoencoder	Image data	Natural scene images from six classes: buildings, forest, glacier, mountain, sea and street.
Turkish Movie Sentiment Analysis Dataset	LSTM	Text data	Turkish movie reviews and rating values. Negative, neutral and positive labels were created from the rating scores.

Kaggle links of the datasets are also given in References section. Main software and libraries used in the Kaggle environment are Python, TensorFlow/Keras, NumPy, Pandas, Matplotlib and scikit-learn. In notebook output TensorFlow version is shown as 2.19.0 and training was done in the Kaggle GPU environment for image data batch size was selected as 32 for text data batch size was selected as 128.

APPLICATION 1: Image Classification with CNN

Problem Definition

Purpose of this application is to predict which class the natural scene images in Intel Image Classification dataset belong to. This is multi class image classification problem. Classes weren't created by me. I used the classes that already exist in Kaggle dataset folder structure. These classes are buildings, forest, glacier, mountain, sea and street. TensorFlow read these folder names as class labels.

I used CNN for image classification because CNN layers can extract features from small parts of an image. In dataset, color, edge, texture and shape information are important for separating the classes.

Dataset and Preprocessing

In CNN application Intel Image Classification dataset was used. In Kaggle environment there are 14,034 images in the train folder and 3,000 images in the test folder. Train folder was split as 80% training and 20% validation. After this split 11,228 images were used for training and 2,806 images were used for validation and test set wasn't used during training.

The images were resized to 128x128 pixels.

Batch size was selected as 32.

Label information was taken automatically from the folder names.

Pixel values were normalized from the 0-255 range to the 0-1 range by using Rescaling(1./255)

The same seed value was used in the train validation split so results could be reproduced.

Model Architecture

CNN model was designed as a simple and understandable structure. Three convolution blocks were used in the model. After each convolution layer, MaxPooling was applied. In this way while the image size became smaller, the model tried to learn more important features. In the last part of the model, Flatten, Dense, Dropout and a softmax output layer were used.

Layer	Description
Input	128x128x3 RGB image input
Conv2D 32 + ReLU	Learns first low level image features.
MaxPooling2D	Reduces the size of the feature map.
Conv2D 64 + ReLU	Extracts more complex features with more filters.
MaxPooling2D	Reduces the image size.
Conv2D 128 + ReLU	Learns higher level features.
MaxPooling2D	Reduces the final feature map size.
Flatten	Converts 2D feature maps into a vector.
Dense 128 + ReLU	Combines the features for classification.
Dropout 0.30	Temporarily turns off some neurons to reduce overfitting.
Dense 6 + Softmax	Produces probabilities for six classes.

Total number of parameters in the model was 3,305,414. As the loss function, sparse_categorical_crossentropy was used because the labels weren't in one hot format. They were

integer class numbers. Adam was selected as the optimizer. Adam is a practical and commonly used optimizer in basic deep learning applications because it adjusts the learning rate adaptively.

Training Settings

Setting	Value	Reason
Epoch	15	It was seen as enough for the model to learn image features. More epochs could increase the risk of overfitting.
Batch size	32	This value was suitable for Kaggle GPU memory and is commonly used in image training.
Loss function	sparse_categorical_crossentropy	It is suitable for multi class classification with integer labels.
Optimizer	Adam	It gives stable and fast learning in beginner level projects.
Train/Val/Test	11228 / 2806 / 3000	Validation was used to follow the training process. Test data was used to measure the final performance.

Results and Graph Comments

The test results of the CNN model are given below. The test accuracy was about 80.67%. This is a reasonable result for a simple CNN model in a scene classification problem with six classes.

Metric	Value
Test loss	1.0416
Test accuracy	0.8067

Class	Precision	Recall	F1-score	Support
buildings	0.74	0.80	0.77	437
forest	0.94	0.96	0.95	474
glacier	0.75	0.78	0.77	553
mountain	0.77	0.75	0.76	525
sea	0.83	0.76	0.79	510
street	0.82	0.80	0.81	501
macro avg	0.81	0.81	0.81	3000
weighted avg	0.81	0.81	0.81	3000

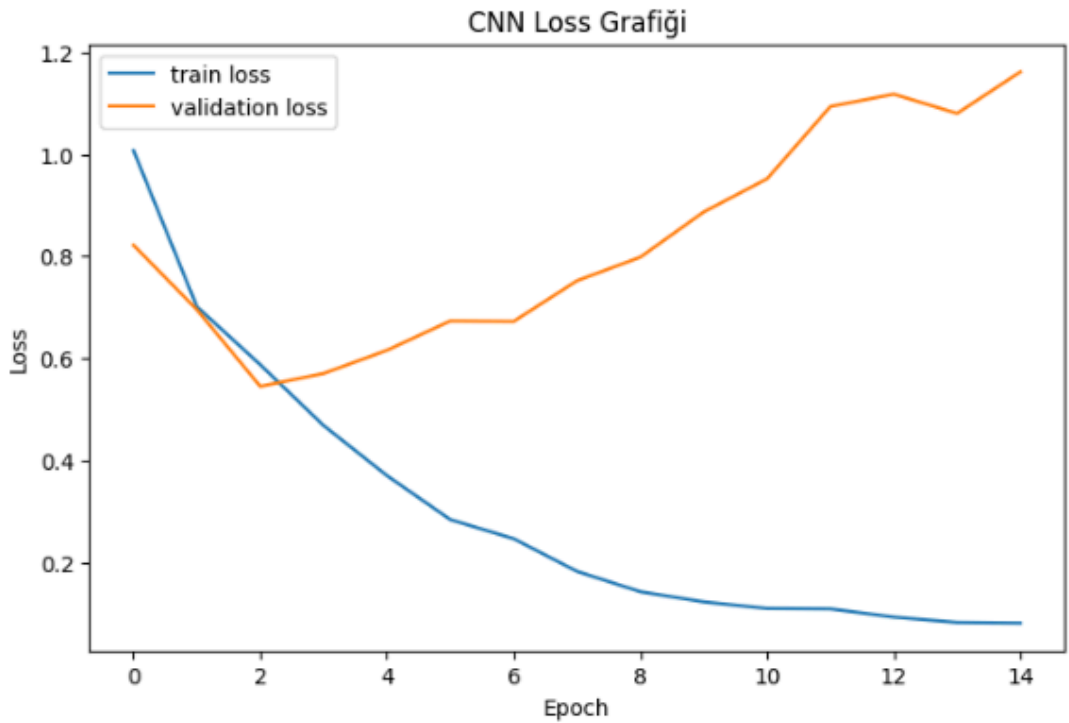


Figure 1. CNN training ve validation loss graph.

When we look at the loss graph train loss decreased regularly. However validation loss started to increase after the first few epochs. This shows that the model learned the training data very well but it's couldn't generalize to the validation data at the same level. So there is an overfitting tendency in the CNN model.

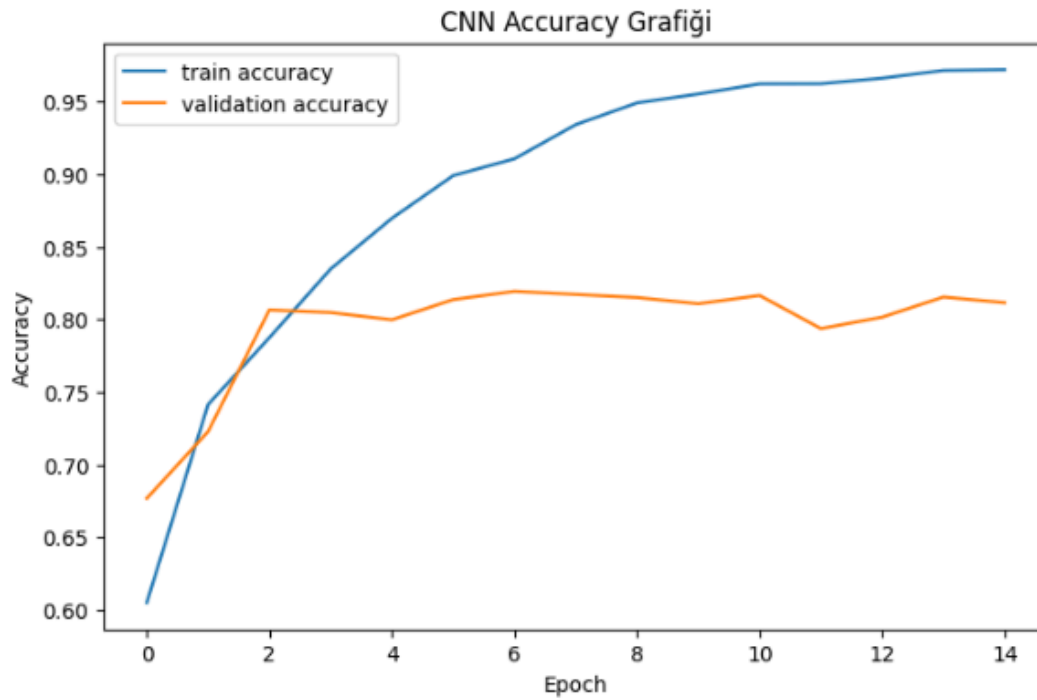


Figure 2. CNN training ve validation accuracy graph.

In the accuracy graph train accuracy increased to about 97%. The validation accuracy stayed around 81%. The difference between train and validation accuracy also supports the overfitting comment. However validation accuracy didn't completely decrease. It stayed stable at a certain level.

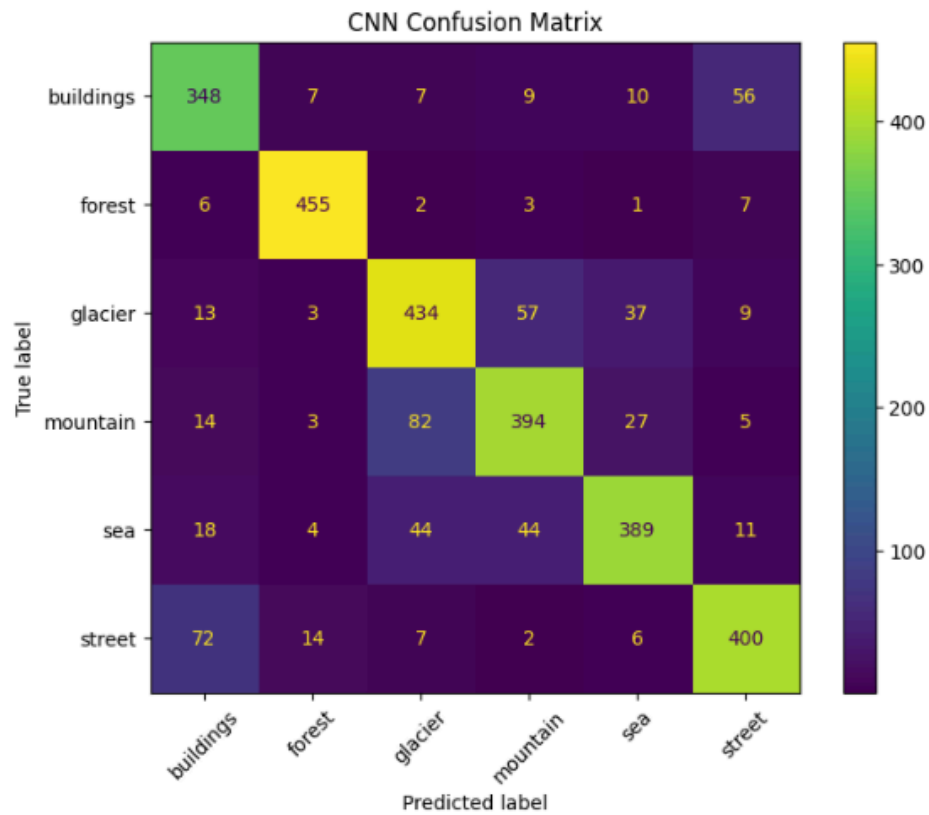


Figure 3. CNN confusion matrix.

When the confusion matrix is examined most successful class of the model is the forest class. This result can be explained because forest images are more clear and different from other classes in terms of color and texture.

There are some mistakes between the glacier and mountain classes. This is expected because these two classes can have similar images such as snow, mountains and open areas. There is also confusion between buildings and street classes. This is because city images can include both buildings and street details at the same time.

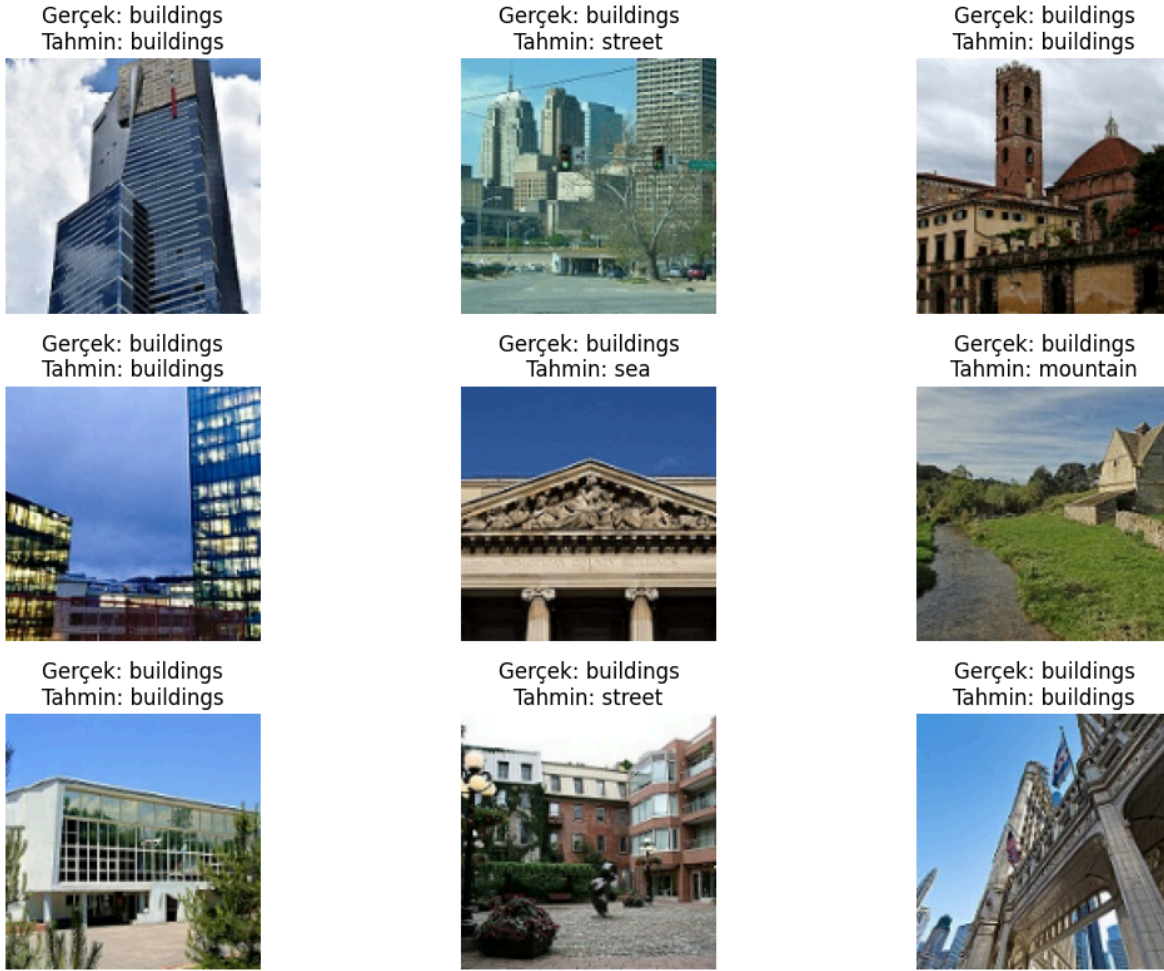


Figure 4. Sample predictions of the CNN model on test images.

In the sample predictions some buildings images were classified correctly. However in some examples, the model predicted them as street or sea. These examples show that this model isn't only sensitive to main object but also to the background and general color information.

APPLICATION 2: Turkish Sentiment Analysis with RNN-Based LSTM

Problem Definition

Goal of this application is to predict sentiment class of Turkish movie reviews. This is a text classification problem with three classes. Classes were created as negative, neutral and positive.

For the Recurrent Neural Networks part in the model list, I used LSTM. LSTM is a model from the RNN family and it is suitable for text data because text is sequential. Order of words in a review can

change the meaning. For this reason LSTM is a more suitable choice for text compared to simple Dense models.

Dataset and Preprocessing

In LSTM application Turkish Movie Sentiment Analysis Dataset was used. This dataset include comment , film_name and point columns. Film name wasn't used directly in model. Only review text and rating value were used.

A total of 83,227 rows were read from the dataset. Since the rating values could come in comma format they were first taken as strings. Then commas were changed to dots and the values were converted to numeric type.

- Column names were changed to lowercase
- Only the comment and point columns were used
- Empty comments and empty point values were cleaned
- Comments were converted to string type and changed to lowercase
- The point value was converted to a numeric value, and labels were created from these scores
- Label structure was: $\text{point} < 2$ negative, $2 \leq \text{point} < 4$ neutral, and $\text{point} \geq 4$ positive
- To make the dataset more balanced a maximum of 15,000 samples were taken from each class
- Text data was converted into numeric sequences by using Keras Tokenizer. Then pad_sequences was used to make the sequence length 80
- To avoid dtype errors input sequences and labels were converted to int32 type

Step	Value / Result
Original row count	83,227
Label distribution after cleaning	negative: 11,531; neutral: 28,360; positive: 43,336
Total reviews after sampling	41,531
Distribution after sampling	negative: 11,531; neutral: 15,000; positive: 15,000

Train / Validation / Test	29,071 / 6,230 / 6,230
Vocabulary size	20,000
Maximum sequence length	80

Model Architecture

LSTM model isn't a very advanced model but it is suitable for text classification. First words were converted into embedding vectors. Then LSTM layer tried to learn the information from the word order. A Dropout layer was added to reduce the risk of overfitting. At end softmax layer produced probabilities for three sentiment classes.

Katman	Açıklama
Input	Token sequence with length 80
Embedding (20000, 64)	Converts word indexes into 64 dimensional vectors.
LSTM 64	Tries to learn the sequence information in the text.
Dropout 0.30	Used to reduce overfitting.
Dense 32 + ReLU	Prepares the LSTM output for classification.
Dense 3 + Softmax	Produces probabilities for negative, neutral, and positive classes.

Total number of parameters in model is 1,315,203. As loss function `sparse_categorical_crossentropy` was used because this is three class classification problem and labels are in integer format. Adam was selected as optimizer.

Training Settings

Setting	Value	Reason
Epoch	6	It was kept at a medium level because long training on text data could cause overfitting.
Batch size	128	Since text data is lighter than image data, a larger batch size could be used
Loss function	<code>sparse_categorical_crossentropy</code>	It is suitable for a three class problem with integer labels.
Optimizer	Adam	It was selected for fast and stable training

Train/Val/Test	29071 / 6230 / 6230	Data splited as about 70%, 15%, and 15%.
----------------	---------------------	--

Results and Graph Comments

LSTM model showed limited performance on Turkish sentiment classification task. Test accuracy was approximately 37.34% which is only slightly higher than random guessing in a three class classification problem. Classification result show model predicted positive class very often. Positive recall was high but negative and neutral recall values were very low and that means model couldn't separate negative and neutral reviews effectively

This limited performance is explainable first sentiment labels weren't manually annotated: they were created from rating scores so some review texts may not fully match their assigned sentiment labels. In addition Turkish is a morphologically rich language. Suffixes, negation structures and different word forms can make sentiment classification difficult for a simple Tokenizer based LSTM model. Third neutral class is naturally difficult because neutral reviews may contain both positive and negative expressions. For these reasons low performance isn't only related to model architecture but also to dataset structure and difficulty of language task

Metric	Value
Test loss	1.1858
Test accuracy	0.3734
Macro avg F1-score	0.23
Weighted avg F1-score	0.25

Class	Precision	Recall	F1-score	Support
Negative	0.46	0.04	0.08	1730
Neutral	0.48	0.05	0.09	2250
Positive	0.37	0.95	0.53	2250
Accuracy			0.37	6230
Macro avg	0.44	0.35	0.23	6230
Weighted avg	0.43	0.37	0.25	6230

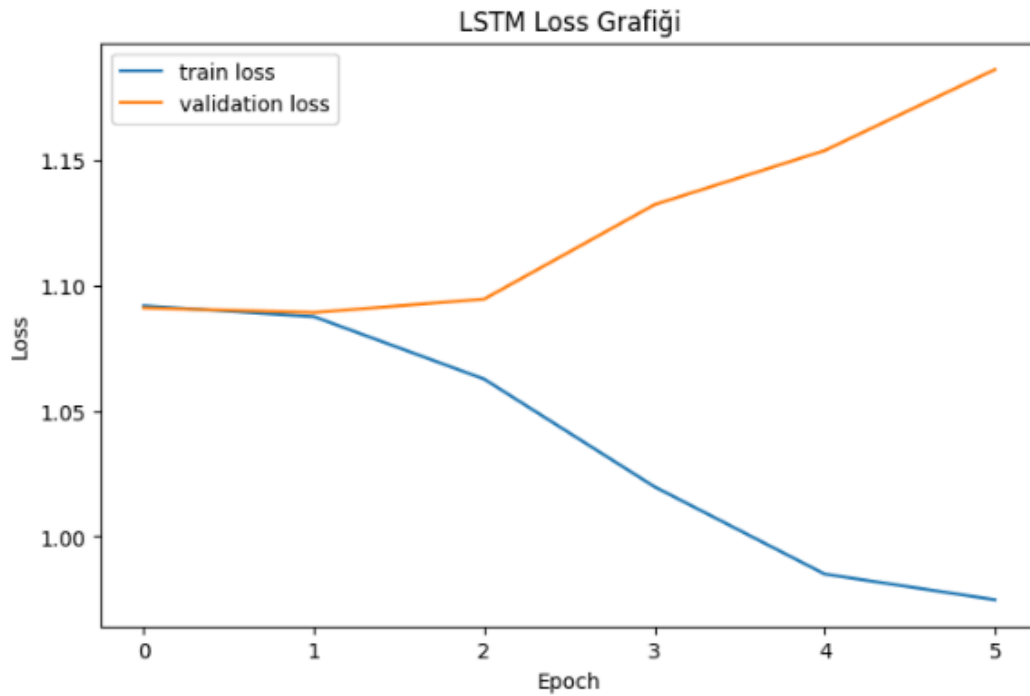


Figure 5. LSTM training ve validation loss graph.

In loss graph training loss decreased during epochs but validation loss increased after first epochs. This tell us model learned some patterns from the training data but it couldn't generalize well to validation data. The gap between training loss and validation loss indicates weak generalization and an overfitting tendency in LSTM model.

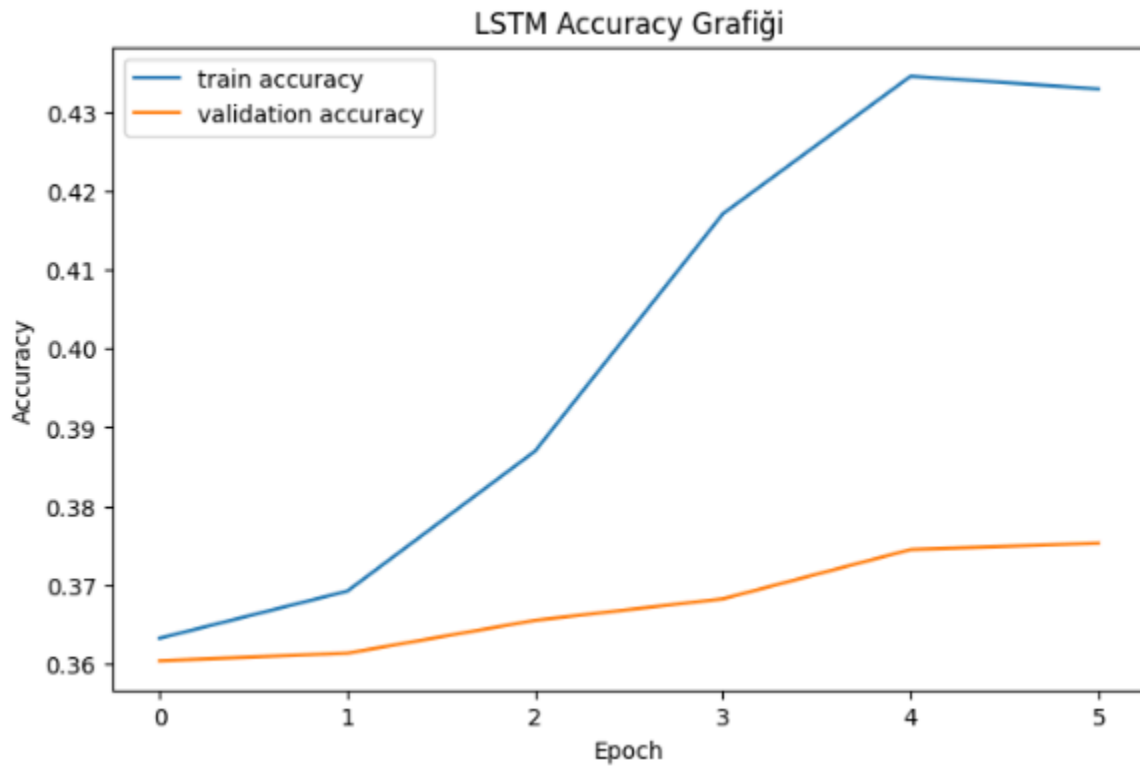


Figure 6. LSTM training ve validation accuracy graph.

Accuracy graph train accuracy increased regularly. Validation accuracy increased clearly in last epoch. However general accuracy level is still low. The reasons for this may be that the reviews are written in natural language, Turkish has many suffixes and different word forms, and the sentiment labels were created from rating scores not directly from the review text.

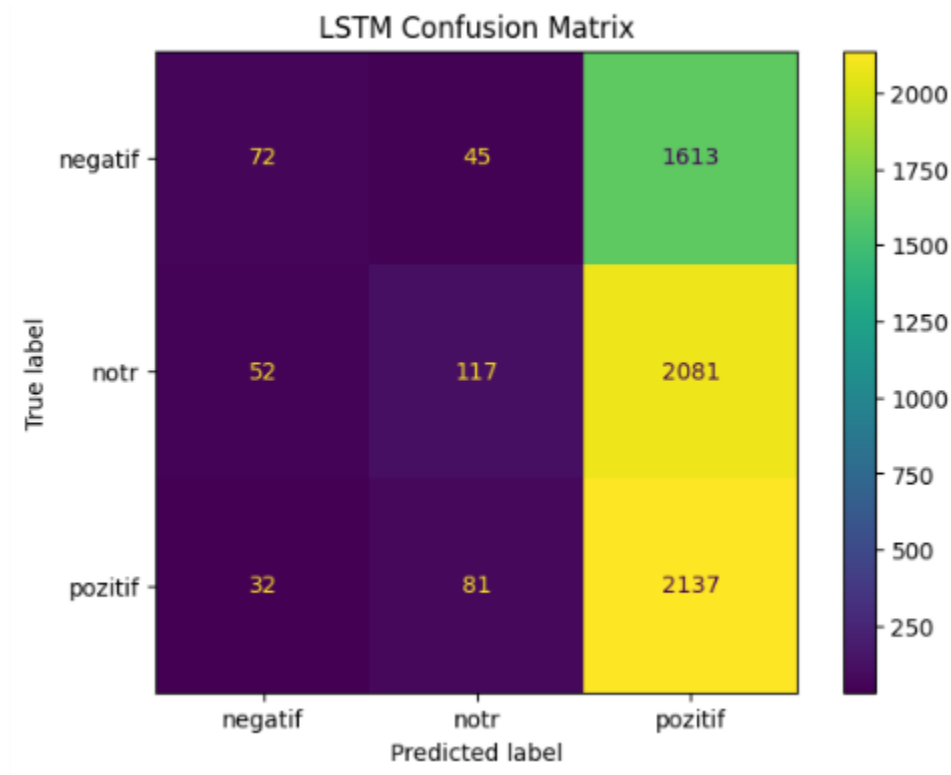


Figure 7. LSTM confusion matrix.

Confusion matrix tell us model had a strong tendency to predict the positive class. Most of real positive samples were classified as positive but many negative and neutral samples were also incorrectly predicted as positive. This explains why positive recall value is high while the recall values of negative and neutral classes are very low. Model learned some positive patterns but it failed to distinguish negative and neutral sentiment expressions clearly.

Review	Model Prediction	Comment
The movie was very good. I liked the acting and the story.	positive	This prediction is meaningful because sentence contains clearly positive expressions
It was very boring. I think it was a waste of time.	positive / wrong prediction	Negative prediction was expected. This error tell us model has difficulty recognizing negative expressions
It was an average movie. Some scenes were good but some scenes were weak.	positive -wrong or uncertain prediction	Neutral prediction could be expected and sentence contains mixed sentiment so model may be confused

From these results LSTM model was able to make basic sentiment separation but it didn't give a very strong performance. Especially neutral class is usually difficult in sentiment analysis problems because neutral reviews can include positive and negative words.

APPLICATION 3: Image Reconstruction with Autoencoder

Problem Definition

Application goal isn't to classify images. Goal is to take an image as an input and produce an output image that is close to the same image for this reason this problem is an image reconstruction problem.

Autoencoder model doesn't use labels like supervised classification models. Input image is also the target image. First model compresses the image into a smaller representation in the encoder part. Then in the decoder part it tries to reconstruct the image from this representation.

Dataset and Preprocessing

The Intel Image Classification dataset was also used for Autoencoder model. Using the same dataset isn't a problem here because the task is different. In CNN part, labeled classification was done. In Autoencoder part reconstruction was done without using labels.

The images were resized to 128x128.

Labels weren't taken by using `label_mode=None`

Pixel values were divided by 255.0 and converted to the 0-1 range.

The format was prepared as (images,images) because the input and target were same image.

The train folder was split as 80% train and 20% validation. Test folder was used for the final evaluation.

Model Architecture

Autoencoder model was built with a convolutional structure. In encoder part image size was reduced by using Conv2D and MaxPooling2D layers. In decoder part image was increased back to 128x128x3 size by using UpSampling2D and Conv2D layers.

In last layer sigmoid activation was used because the images were normalized to the 0-1 range.

Part	Layers	Task
Encoder	Conv2D 32 + MaxPooling, Conv2D 64 + MaxPooling, Conv2D 128 + MaxPooling	Reduces the image into a smaller and dense representation
Decoder	Conv2D 128 + UpSampling, Conv2D 64 + UpSampling, Conv2D 32 + UpSampling	Rebuilds the image from the compressed representation by increasing its size
Output	Conv2D 3 + Sigmoid	Produces the RGB image in the 0-1 range

Total number of parameters in model was 333,955. Loss function mean squared error (MSE) was used and this because the aim isn't class prediction. My goal is to reduce pixel difference between the original image and the reconstructed image. Adam was used again as the optimizer.

Training Settings

Setting	Value	Reason
Epoch	20	Since a reconstruction problem can improve more slowly than classification a little more epoch was used compared to the CNN model
Batch size	32	This is a balanced value for GPU memory when working with image data
Loss function	MSE	It measures the pixel difference between the original and reconstructed image
Optimizer	Adam	It gives simple and stable optimization
Train/Val/Test	11228 / 2806 / 3000	Same images were used as input and target without using labels

Results and Graph Comments

In Autoencoder model accuracy wasn't used because this isn't a classification problem. Instead reconstruction loss was followed. MSE value was used as the reconstruction loss. Test reconstruction loss was found as about 0.0043

Metric	Value
Last epoch train loss	0.0043
Last epoch validation loss	0.0043
Test reconstruction loss	0.0043

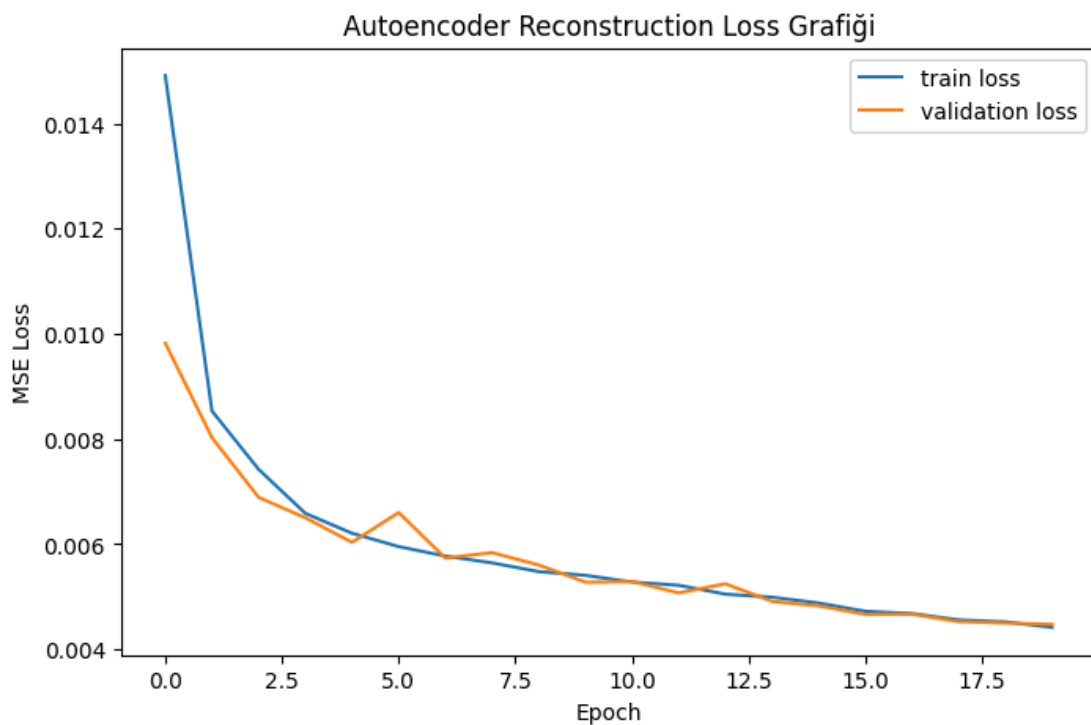


Figure 8. Autoencoder reconstruction loss graph.

In the loss graph, both train loss and validation loss decreased regularly. Also, the two lines were very close to each other. This shows that the Autoencoder model didn't only memorize the training data. It also showed similar reconstruction success on the validation data.

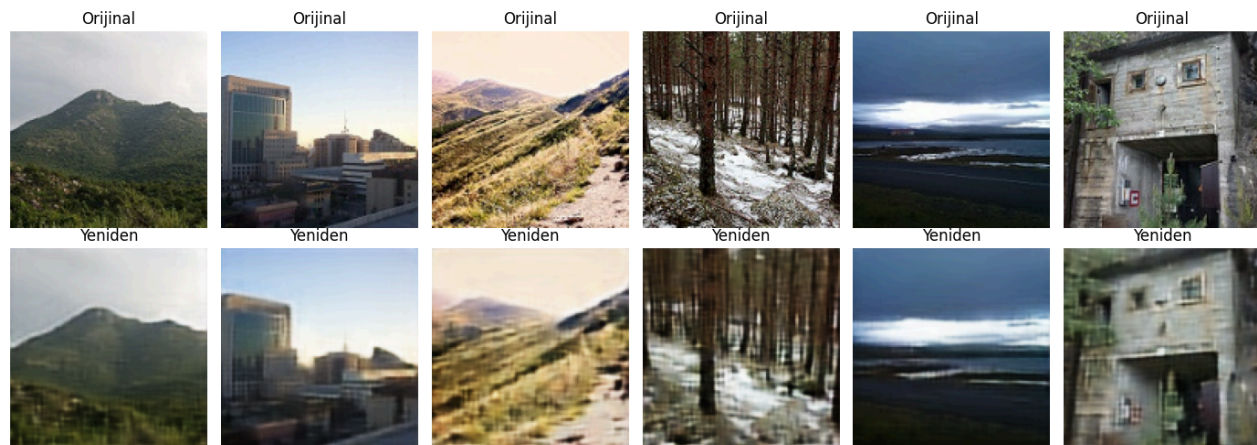


Figure 9. Comparison of the original and reconstructed images in the Autoencoder model.

In visual comparison it can be seen that the model could keep the general colors, big shapes and scene structure of images. In contrast the reconstructed images are more blurry than the original images. This is an expected result because some small details can be lost when the image is compressed into a smaller representation in the encoder part. Model kept the general structure well but it was limited in fine edges and small details.

General Discussion

When three models are evaluated together, it is seen that each model behaved differently according to its own problem type. CNN model gave the highest accuracy in image classification. Reason for this is that CNN architecture is directly suitable for learning image features. In contrast difference between train and validation graphs shows that there is overfitting in model.

LSTM model produced weakest result among the three applications but this doesn't mean that experiment failed. Result tell us difficulty of sentiment classification on Turkish movie reviews. The labels were created from rating scores and this may introduce label noise because a rating score may not always directly reflect the sentiment expressed in the review text. In addition Turkish has many suffixes and word variations which makes it harder for a simple Tokenizer based LSTM model to learn sentiment patterns. Confusion matrix also tell us model was biased toward the positive class and couldn't separate negative and neutral reviews effectively.

In Autoencoder model reconstruction quality was evaluated instead of classification accuracy. Low loss value and visual outputs keeping general scene structure show that the model worked for its purpose. Blurry images are caused by the loss of small details while model compresses image.

Model	Problem Type	Data Type	Main Metric	Result Comment
CNN	6 class image classification	Image	Accuracy: 0.8067	Generally successful but there is overfitting.
LSTM	3 class Turkish sentiment analysis	Text	Accuracy: 0.3734	It learned at a basic level but it had difficulty with the neutral class and mixed reviews.
Autoencoder	Image reconstruction	Image	MSE: 0.0043	It kept the general structure but there is blurriness in small details.

What Can Be Done for Better Results?

Data augmentation can be added to CNN model. In that way model can become more resistant to different angles, zooming, and small changes in images.

For CNN model dropout rate or size of the Dense layer can be tested again. Since there is overfitting regularization can be increased.

For LSTM model manually labeled sentiment data could improve the reliability of the labels. More advanced preprocessing methods can be used for Turkish text such as removing noisy characters, handling suffixes more carefully and using Turkish specific tokenization. Pretrained Turkish word embeddings or transformer based models could also improve performance because they can represent Turkish language structure better than a simple Tokenizer based LSTM model.

For Autoencoder model, a deeper decoder or structures similar to skip connections can help to keep image details better but this also makes the model more complex.

Conclusion

In my project three different deep learning models were applied by using Python and TensorFlow/Keras. Image classification was done with CNN model, text classification was done with LSTM model, and image reconstruction was done with Autoencoder model. Project met requirements of using three different models, text data and image data.

CNN model gave the best classification result on the Intel Image Classification dataset with about 81% test accuracy. LSTM model reached about 37% accuracy on Turkish movie reviews. Although this performance was limited it provided important result for analysis. Model had difficulty separating sentiment classes especially negative and neutral reviews. This limitation can be explained by label noise from rating based labels morphological complexity of Turkish language and the simple LSTM architecture used in project. Autoencoder model reconstructed the general structure of images with test reconstruction loss of approximately 0.0043.

In general project showed how different deep learning models can be used for different data types and problem types. Most important result is looking only at a high accuracy value isn'tt enough. When loss graphs, validation results, confusion matrix, classification report and sample outputs are evaluated together strong and weak sides of the models can be understood more clearly.

Sources

1. Intel Image Classification Dataset. Kaggle:
<https://www.kaggle.com/datasets/puneet6060/intel-image-classification>
2. Turkish Movie Sentiment Analysis Dataset. Kaggle:
<https://www.kaggle.com/datasets/mustfkeskin/turkish-movie-sentiment-analysis-dataset>